

1. Introducción del Proyecto

La culminación de un proceso formativo en el área de programación de algoritmos requiere que el estudiante demuestre la integración de todos los conceptos y estructuras aprendidos a lo largo del ciclo académico. El presente desafío final tiene como propósito evaluar la capacidad del estudiante para diseñar, desarrollar y documentar un sistema de software funcional, coherente y bien estructurado, utilizando el lenguaje de programación C# y aplicando de manera obligatoria la totalidad de los contenidos abordados en las guías de estudio.

El proyecto consiste en el desarrollo de un Sistema Integral de Gestión de Biblioteca Universitaria, una solución informática que responde a una necesidad real dentro del ámbito educativo salvadoreño. Este sistema permitirá gestionar el registro de libros, usuario. Manteniendo la persistencia de datos mediante archivos. La temática seleccionada garantiza la aplicación natural de todas las estructuras programáticas estudiadas durante el ciclo.

Este desafío se realizará en PAREJAS. No se aceptarán trabajos duplicados. Todo proyecto sospechoso de copia será anulado automáticamente y sujeto a proceso académico disciplinario.

2. Objetivo General

Desarrollar un sistema de software funcional denominado Sistema Integral de Gestión de Biblioteca Universitaria utilizando el lenguaje de programación C#, que integre de forma obligatoria y correcta todas las estructuras de programación estudiadas durante el ciclo académico: estructuras secuenciales, de decisión (if-else y switch-case), de repetición (for, while y do-while), vectores, matrices, cadenas, structs y manejo de archivos; con el fin de demostrar la capacidad del estudiante para resolver problemas reales mediante la construcción de sistemas de software bien estructurados, documentados y funcionales.

2.1 Objetivos Específicos

- Implementar correctamente las estructuras secuenciales, de decisión y de repetición en los distintos módulos del sistema.
 - Utilizar vectores y matrices para el almacenamiento y procesamiento eficiente de datos en memoria.
 - Aplicar el manejo de cadenas para validación, búsqueda y manipulación de información textual.
 - Definir y utilizar estructuras (struct) como tipos de datos personalizados para modelar entidades del dominio.
 - Implementar la persistencia de datos mediante la lectura y escritura de archivos de texto plano (.txt) y valores separados por coma (.csv).
 - Desarrollar una segunda versión del sistema utilizando herramientas de Inteligencia Artificial y realizar un análisis comparativo entre ambas implementaciones.
-

3. Descripción del Sistema

El Sistema Integral de Gestión de Biblioteca Universitaria de la Universidad Don Bosco, es una aplicación de consola desarrollada en C# que permitirá al usuario administrar de forma completa los recursos y operaciones de una biblioteca universitaria. El sistema operará mediante un menú interactivo de texto y deberá ser capaz de gestionar los siguientes módulos:

3.1 Módulos del Sistema

Módulo A – Gestión de Libros

- Registro de nuevos libros con los campos: código (string), título (string), autor (string), editorial (string), año de publicación (int), categoría (string), cantidad de ejemplares disponibles (int).
- Búsqueda de libros por código.
- Listado completo de libros registrados.
- Eliminación de un libro.

Módulo B – Gestión de Usuarios

- Registro de usuarios con los campos: carné (string), nombre completo (string), carrera (string), correo electrónico (string), teléfono (string), estado (activo/inactivo).
- Búsqueda de usuarios por carné o por nombre.
- Listado de todos los usuarios registrados.

Módulo C – Gestión de Préstamos

- Registro de préstamo vinculando carné de usuario con código de libro.
- Validación de disponibilidad del libro antes del préstamo.
- Registro de fecha de préstamo y fecha estimada de devolución (como cadenas de texto en formato dd/mm/yyyy).
- Registro de la devolución de un libro y actualización del inventario.
- Consulta del historial de préstamos activos de un usuario.
- Actualizar estado del préstamo

Manejo de Archivos

- Carga automática de datos desde archivos al iniciar el sistema.
- Guardado automático de datos en archivos al cerrar el sistema o al ejecutar una operación de guardado manual.

4. Requerimientos Funcionales Detallados

4.1 Menú Principal

El sistema deberá presentar un menú principal numerado con las siguientes opciones al iniciar:

1. Gestión de Libros
 2. Gestión de Usuarios
 3. Gestión de Préstamos
 4. Salir del Sistema
-

Cada opción del menú principal deberá desplegar un submenú con las operaciones específicas del módulo correspondiente.

4.2 Requerimientos de Almacenamiento en Memoria

- El sistema deberá mantener como máximo 10 libros, 5 usuarios y 10 préstamos simultáneamente en los arreglos en memoria.
- Los datos deberán almacenarse en arreglos de structs.

4.3 Requerimientos de Validación

- Código de libro: formato alfanumérico de exactamente 8 caracteres (ej. LIB00001).
- Carné de usuario: formato numérico de exactamente 8 dígitos.
- Correo electrónico: debe contener el carácter '@' y un punto posterior al mismo.
- Año de publicación: entre 1900 y el año actual.
- Cantidad de ejemplares: valor entero no negativo.
- Fechas: formato dd/mm/yyyy validado por longitud y separadores.
- Todos los campos de texto obligatorios deben ser verificados para evitar cadenas vacías o nulas.
- El sistema debe capturar excepciones de formato al leer valores numéricos desde consola.

4.4 Requerimientos de Archivos

El sistema manejará los siguientes archivos ubicados en la carpeta Data/ del proyecto:

Archivo	Tipo	Descripción
libros.csv	CSV	Almacena todos los libros registrados
usuarios.txt	TXT	Almacena todos los usuarios registrados
prestamos.txt	TXT	Almacena el historial de préstamos

6. Restricciones del Proyecto

6.1 Restricciones Tecnológicas

- El proyecto debe desarrollarse exclusivamente en C# usando .NET (versión 6 o superior recomendada).
 - La aplicación debe ser de consola (Console Application). No se aceptarán interfaces gráficas (WinForms, WPF, MAUI, etc.).
-

6.2 Restricciones de Autoría

- El trabajo es en pareja o individual. La colaboración en código es causa de anulación.
- Se aplicará detección de similitud entre proyectos entregados. Dos proyectos con más del 70% de similitud en código serán anulados sin excepción.
- El estudiante debe ser capaz de explicar cualquier línea de código presente en su proyecto durante la presentación del video.

6.3 Restricciones de Entrega

- La entrega es únicamente mediante repositorio Git (GitHub o GitLab). No se aceptan archivos comprimidos enviados por enlace en plataforma virtual.
- El repositorio debe ser público para permitir la revisión del docente.
- La fecha límite de entrega es inapelable. No se aceptarán entregas tardías bajo ninguna circunstancia.

7. Formato de Entrega mediante Repositorio Git

7.1 Plataforma

El estudiante deberá crear un repositorio en GitHub o GitLab con el nombre en formato:

DesafioFinal_[Carné1]_[Carné2]

Ejemplo: DesafioFinal_CM210345_BH160107

7.2 Historial de Commits

El repositorio debe contar con un mínimo de 10 commits a lo largo del desarrollo, con mensajes descriptivos en español. Se evaluará que los commits reflejen un progreso gradual y no una entrega de una sola vez.

Ejemplo de mensaje de commit correcto: 'feat: implementar módulo de registro de libros con validación de código'

Ejemplo de mensaje INCORRECTO: 'subiendo archivos', 'cambios', 'listo'

7.3 Enlace de Entrega

El estudiante deberá compartir el enlace del repositorio en el formulario oficial de entrega habilitado en la plataforma virtual del curso antes de la fecha indicada **26 de MAYO 2029**.

8.1 Contenido del README.md

El archivo README.md debe incluir obligatoriamente los siguientes apartados:

1. Nombre completo del estudiante y carné.
2. Instrucciones para clonar y ejecutar el proyecto.
3. Enlace directo al video demostrativo.

9. Instrucciones para la Grabación del Video Demostrativo

9.1 Especificaciones Técnicas

Especificación	Requisito
Duración mínima	15 minutos
Duración máxima	25 minutos
Resolución mínima	1280 × 720 píxeles (HD)
Audio	Voz del estudiante audible y clara en todo el video
Cara visible	Cámara encendida durante la presentación inicial (mínimo 2 minutos)
Plataforma	YouTube (no listado) o Google Drive compartido como enlace público
Formato	MP4 recomendado

9.2 Contenido Obligatorio del Video

El video debe seguir el siguiente orden y demostrar cada apartado de forma clara:

Segmento 1 – Presentación Personal (2–3 min)

- Presentarse en cámara: nombre completo, carné y nombre de la materia.
- Explicar brevemente la problemática que resuelve el sistema desarrollado.
- Mostrar la estructura del repositorio en pantalla.

Segmento 2 – Explicación del Código Fuente (5–7 min)

- Mostrar y explicar la definición de los structs (Libro, Usuario, Prestamo).
- Explicar la lógica de al menos un módulo completo (a elección del estudiante).
- Señalar explícitamente dónde se utilizan: if-else, switch-case, for, while, do-while, vectores, matrices, cadenas y archivos.
- Mostrar las validaciones implementadas en código.

Segmento 3 – Ejecución Completa del Sistema (5–8 min)

- Ejecutar el sistema y demostrar el menú principal.
-

- Realizar al menos un registro completo de libro, usuario y préstamo.
- Demostrar la búsqueda de libros y usuarios.
- Ejecutar el registro de una devolución y verificar actualización del inventario.
- Mostrar al menos dos reportes generados.
- Demostrar la exportación de un reporte a archivo .txt.
- Cerrar el sistema y reabrir para demostrar que los datos persisten correctamente desde los archivos.

Segmento 4 – Versión Desarrollada con IA (3–5 min)

- Mostrar el archivo `prompt_utilizado.txt` con el prompt ingresado a la herramienta de IA.
- Mostrar el código generado por la IA y explicar su funcionamiento brevemente.
- Ejecutar la versión generada por la IA y mostrar su funcionamiento.
- Realizar una comparación verbal entre la versión propia y la versión de IA, señalando diferencias, ventajas y limitaciones de cada una.

10. Implementación usando Inteligencia Artificial

10.1 Descripción

Como parte del desafío final, el estudiante deberá desarrollar una segunda versión del sistema utilizando herramientas de Inteligencia Artificial Generativa, tales como ChatGPT, Claude, Copilot u otras herramientas de generación de código. Esta segunda versión tiene como propósito fomentar la reflexión crítica sobre el uso responsable de la IA en el desarrollo de software.

10.2 Entregables Requeridos de la Versión IA

a) Archivo: `prompt_utilizado.txt`

Debe contener el o los prompts exactos que el estudiante ingresó a la herramienta de IA. El prompt debe ser lo suficientemente completo para describir el sistema a desarrollar. Se recomienda elaborar un único prompt detallado que especifique claramente:

- El tipo de sistema a desarrollar.
- Las estructuras de programación que deben aparecer obligatoriamente.

b) Archivo: `codigo_generado.cs`

Debe contener el código completo generado por la herramienta de IA, sin modificaciones. Si la IA generó el código en partes, se deben consolidar todas las partes en este único archivo.

c) Archivo: `evidencia_funcionamiento`

En el video presentar un espacio que evidencie que el código generado por la IA compila y ejecuta correctamente y una breve explicación en el README.

IMPORTANTE: El uso de IA para desarrollar la versión PRINCIPAL del proyecto (versión manual) constituye una violación a la honestidad académica y será tratada como copia. La versión desarrollada con IA es un componente separado y claramente delimitado.

11. Rúbrica de Evaluación

La evaluación del desafío final se realizará con base en los siguientes criterios. Cada criterio se califica en cuatro niveles: Excelente (100% del peso), Bueno (75%), Regular (50%) e Insuficiente (0%).

Criterio de Evaluación	Excelente (100%)	Bueno (75%)	Regular (50%)	Insuficiente (0%)	Peso
1. Estructuras de Control (Secuencial, Decisión y Repetición)	Secuencias, if-else, switch-case, for, while y do-while implementados correctamente en todos los módulos, sin errores lógicos ni ciclos infinitos.	La mayoría de estructuras correctas; algún tipo de ciclo o decisión ausente o con error menor.	Solo algunas estructuras presentes; errores frecuentes en condiciones o selección de estructura.	Estructuras ausentes o con fallos que impiden la ejecución del sistema.	25%
2. Vectores, Matrices y Cadenas	Arreglos de structs, matrices y operaciones de string (búsqueda, comparación, formato) usados correcta y eficientemente.	Vectores y cadenas correctos; matriz presente con uso básico o algún error menor.	Solo vectores o solo cadenas implementados; matriz ausente o incorrecta.	Ninguna de estas estructuras implementada correctamente.	20%
3. Estructuras (struct)	Structs Libro, Usuario y Prestamo bien definidos (mín. 6 campos cada uno), utilizados en todos los módulos del sistema.	Structs definidos y funcionales, con algún campo faltante o subutilización menor.	Al menos un struct definido pero con campos insuficientes o uso limitado.	Structs ausentes o declarados incorrectamente.	15%
4. Manejo de Archivos	Lectura y escritura en .txt y .csv funcionan correctamente. Datos persisten al cerrar y reabrir el sistema. Exportación de reporte operativa.	Lectura y escritura funcionales con algún error menor en persistencia o exportación.	Solo lectura o solo escritura implementada; persistencia parcial.	Manejo de archivos ausente o con errores que impiden la persistencia de datos.	20%
5. Funcionalidad y Validaciones	Sistema completo y sin errores: menú,	Sistema mayormente funcional; algún	Funcionalidad limitada a 2-3 módulos;	Sistema no ejecuta o carece de validaciones	10%

Criterio de Evaluación	Excelente (100%)	Bueno (75%)	Regular (50%)	Insuficiente (0%)	Peso
	submenús, todas las operaciones y validaciones robustas en todos los campos.	módulo incompleto o validaciones parciales.	validaciones mínimas o inconsistentes.	y módulos esenciales.	
6. Calidad del Código y Git	Código indentado, comentado y con nombres descriptivos. Repositorio con 10+ commits descriptivos, estructura correcta y README completo.	Código ordenado con leves inconsistencias. Repositorio con commits y estructura aceptable.	Código desorganizado pero legible. Pocos commits o README incompleto.	Código ilegible. Sin repositorio o entrega incorrecta.	5%
7. Implementación con IA y Video	Segunda versión funcional, prompt detallado, comparación reflexiva documentada. Video claro (15-25 min) que evidencia todo el sistema y el código.	Versión IA funcional con comparación básica. Video completo con alguna omisión menor.	Código IA generado sin análisis comparativo. Video incompleto o con calidad deficiente.	Versión IA o video ausentes, o sin relación con el proyecto.	5%
TOTAL					100%

¡Mucho éxito en tu desafío final! Recuerda: la constancia y la práctica son las únicas claves del aprendizaje.